

1. Preparation of Source codes

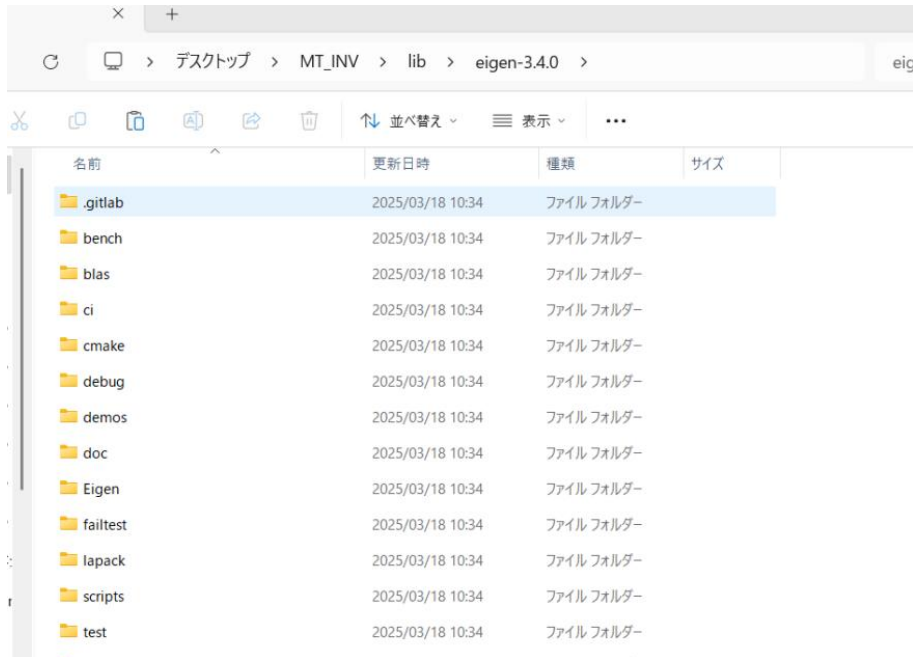
- Download the source code from the URL below:

<https://github.com/SuzukiAtsushi19911107/FV3DMT>

名前	状態	更新日時	種類	サイズ
optimlib	🟢	2025/12/23 16:16	ファイル フォルダー	
sample	🟢	2025/12/23 16:16	ファイル フォルダー	
tools	🟢	2025/12/23 16:16	ファイル フォルダー	
Analysis.h	🟢	2025/12/23 16:15	C/C++ Header	12 KB
BiCGSafe.h	🟢	2025/12/23 16:15	C/C++ Header	8 KB
Boundary.h	🟢	2025/12/23 16:15	C/C++ Header	1 KB
ConstantValues.h	🟢	2025/12/23 16:15	C/C++ Header	1 KB
DivergenceCorrection.h	🟢	2025/12/23 16:15	C/C++ Header	3 KB
Element.h	🟢	2025/12/23 16:15	C/C++ Header	13 KB
FineGrainedILU.h	🟢	2025/12/23 16:15	C/C++ Header	2 KB
Function.h	🟢	2025/12/23 16:15	C/C++ Header	1 KB
Functions.h	🟢	2025/12/23 16:15	C/C++ Header	22 KB

- Download Eigen library from the URL below:

<https://eigen.tuxfamily.org/>



- Optimlib, which is used for optimization in the program is located in the downloaded items.

NOTE:: You MUST use the Optimlib in the downloaded items, not original Optimlib. The original one cannot work as far 2023!!!!!!

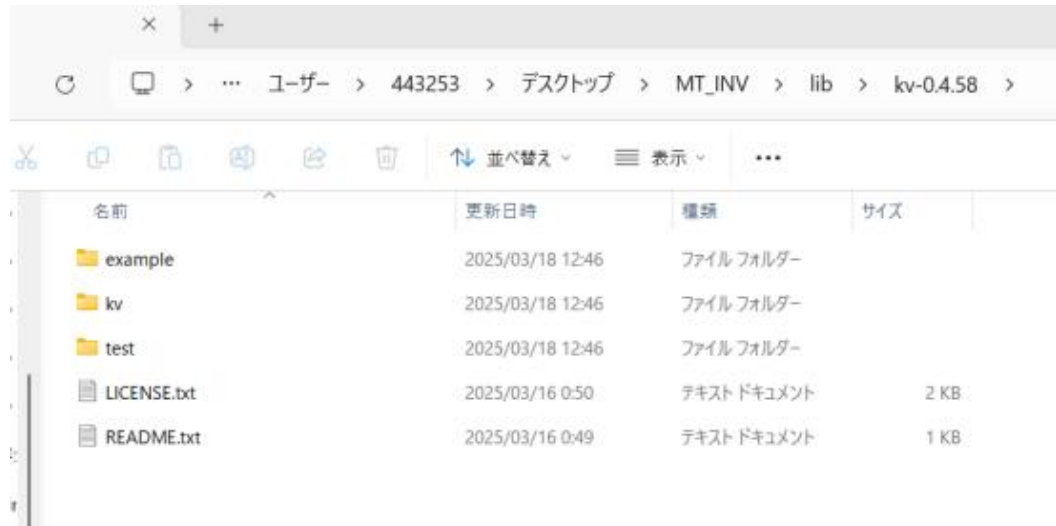
- Intall Boost. Please refer to the URL below. You may need to install Visual Studio, if it is not needed, you do not need to install it.

<https://www.boost.org/users/download/#live>

<https://www.kkaneko.jp/tools/win/boost.html> (in Japanese)

- Download “kv” from the URL below:

<http://verifiedby.me/kv/>



- Install “make” if you need it. You can refer to the URL below:
https://qiita.com/BARANCE_TW/items/c7ffdb311df84d47bddd (in Japanese)
- If you use intel Compiler, please install oneAPI from the below URL:
<https://www.intel.com/content/www/us/en/developer/tools/oneapi/base-toolkit-download.html>
- Set Path to the compiler. As default, the paths are: “C:\Program Files (x86)\Intel\oneAPI\compiler\2025.0\bin” and “C:\Program Files (x86)\Intel\oneAPI\compiler\2025.0\lib”

- Set Paths of the libraries in makefile:

```

1 CC = icpx
2 FLAGS = -qopenmp -O3 -xhost
3 INCB = -I C:\boost\build\include\boost-1_88
4 INCE = -I C:\Users\xxxxx\Desktop\MT_INV\lib\eigen-3.4.0
5 INCO = -I C:\Users\xxxxx\Desktop\MT_INV\lib\optimlib\header_only_version
6 INCK = -I C:\Users\xxxxx\Desktop\MT_INV\lib\kv-0.4.58
7 OUT = MT_INV.exe
8 MT_INV: MT.o Analysis.o BiCGSafe.o Boundary.o DivergenceCorrection.o Element.o Fv
9 $(CC) -o $(OUT) MT.o Analysis.o BiCGSafe.o Boundary.o DivergenceCorrection.o
10
11 MT.o: MT.cpp
12 $(CC) -c MT.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
13 Analysis.o: Analysis.cpp
14 $(CC) -c Analysis.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
15 BiCGSafe.o: BiCGSafe.cpp
16 $(CC) -c BiCGSafe.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
17 Boundary.o: Boundary.cpp
18 $(CC) -c Boundary.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
19 DivergenceCorrection.o: DivergenceCorrection.cpp

```

- Open command prompt and “make”.
- You will get an executable file: MT_INV.exe
- You can also compile with Visual Studio.
- If you use icpx, you may have a warning to notice the deprecation of `omp_set_nested()`. Then, please change the lines as below:

```

if (omp_get_max_threads() >= 2 * analysis->boundary->omega.size())
{
    omp_set_nested(1);
}
else {
    omp_set_nested(0);
}
↓ ↓ ↓
if (omp_get_max_threads() >= 2 * analysis->boundary->omega.size())
{
    omp_set_max_active_levels (2);
}
else {
    omp_set_max_active_levels (1);
}

```

- In some environments, memory allocations make the program so slow. In that case, please use “mimalloc” for the memory management. Please google it for how to use.